

摘要

論文提及，SSD 有潛力改變儲存系統的局勢，然而，關於它們內部結構或 SSD 製造商在追求最佳效能時所面臨的設計選擇的已發表研究仍然很少，因此這篇論文主要列舉了 SSD 的 trade-offs，並且將真實系統所得到的 workload 丟進改良後的硬碟模擬器從而去分析不同 SSD 配置的預期效能。

首先，作者列出了與 NAND-flash solid-state storage 相關的設計 trade-offs，並在論文後半部使用 trace-based disk simulator 來對這些 trade-offs 分析，由於現有 SSD 的詳細內部結構沒法得知(公司的財產)，所以作者會依據模擬後的結果來推敲各個廠牌的 SSD 在設計上的不同考量，從而造成了模擬結果上的差異。

以下是 SSD 設計上的 trade-offs：

1. Data placement：

Data 在 SSD 內部的擺放策略不但會影響 load-balancing，也會影響每個 block 的磨損平攤，因為 data 通常會有 hot cold 的特性(頻繁更新 or 不頻繁更新)，因此如何區分 Data 特性並考慮擺法是至關重要的。

2. Parallelism

論文提及，任何 flash chip 的 bandwidth 和 operation rate 都不足以達到 optimal performance，因此 memory components 需要實現 Parallelism。

我的理解是，單一 chip 能提供的 performance 實際上不夠快，但 SSD 整體能快主要是靠多顆 chip 的平行處理，從而達到遠超單一 chip 的可能好幾倍速率。

3. Write ordering

由於 SSD 的硬體限制，寫入(program)和清除(erase)的細粒度不同，寫入是對 page 而清除則是 block(很多個 page)，因此對於 random writes 類型的 workload 要如何決定寫入順序(或寫入的位址)，是一項難題。

4. Workload management

workload 的特性基本上會直接決定 SSD 最終跑出來的效能評分，因為 SSD 對於資料寫入的決策不會是完美的，對於 sequential workload 能跑出高效能，有可能就會在非 sequential workload 中得到非常差的效能。

除此之外，論文也介紹了當時 SSD 的三大主流類別：

1. Consumer portable storage

最便宜，通常是 USB flash 採用的，random write 效能很差。

2. Laptop disk replacements

中階版，random read 和 write 都比前者好。

3. Enterprise/database accelerators:

最神秘的版本(細節不公開)，同時也是最貴的，但整體效能表現最棒。通常是高階伺服器主機採用。

論文除了針對前面提到的四項 trade-offs 去模擬並分析以外，也順帶討論了 SSD wear-leveling 機制並提出了 greedy strategy 的修改版。

實驗方法與精神

該論文利用現實情境中得到的 workload(如 TPC-C, Exchange, IOzone 與 Postmark)，展示了 SSD 在隨機寫入和數據讀寫順序的變化如何影響其 performance，裡面也簡單介紹了各個 workload 的不同(例如 read 和 write 的比例)。

論文除了使用各種不同的 workload 來完整重現實際情境以外，也針對 SSD 採用了不同的配置來進行模擬，包括 parallelism 採用的策略、logical block size 的選擇、是否採用 interleaving 等等，非常的詳細。

最後，作者針對了五種設計策略的優缺點進行說明，直接上圖：

Technique	Positives	Negatives
Large allocation pool	Load balancing	Few intra-chip ops
Large page size	Small page table	Read-modify-writes
Overprovisioning	Less cleaning	Reduced capacity
Ganging	Sparser wiring	Reduced parallelism
Striping	Concurrency	Loss of locality

Wear-leveling 介紹

首先，Garbage collection 和 Wear-leveling 之間會有取捨，若採用高效能的 GC，儘管可以減少整體 SSD 的磨損程度，但是對於 Block 的壽命是有害的，因為高效的 GC 通常造成了不均勻的 Block 磨損。

例如採用 Greedy 的策略(每次都清除能得到最多 free page 的那些 block)有個缺點，就是會常常對那些更新頻繁的資料(hot data)所占用的 block 進行清除，而這些 block 往往都在相同的區塊，會被重複使用並清除，造成僅有一部份的 blocks 壽命會迅速下降，而另一部分 cold data 的 block 則幾乎不會被清除。

因此作者提出了改良版策略，每個 block 會記錄自己的 remaining lifetime，若某個 block 的 remaining life time 低於 average remaining lifetime 的 retirement age，就代表該 block 壽命快要到極限了，我們就將位於 migration-candidate queue 的 cold data 搬到該 block 放置，來降低 block 被 GC 選上的機率。

後面則是針對該 wear-leveling 策略進行了實驗，結論是比 Greedy 策略平均磨損表現還要好很多，對於 Greedy 會產生 223 個 Expired blocks 的情況，該篇論文的策略只會有 0 個 Expired blocks，但這也是從 GC 效能中做了一些取捨，有 4.7% 的 overhead to the average I/O operation latency。

結論

這篇論文詳盡說明了 SSD 在設計上的各種 trade-offs，以及當時三種主流 SSD 之間的差別。也提供了豐富的 background 來讓讀者能熟悉 SSD 的知識。後面作者採用了真實情境的 workload 丟入 disk simulators 來分析不同 SSD 配置上的效果。最後提出了一種 wear-leveling 機制，能夠很好地均攤 block 之間的磨損程度。

值得注意的點

由於實驗皆是依靠模擬運行，這些結果也都是基於假設好的 SSD 配置，因此實際硬體表現可能還是會有不同，若能夠有一些真實的 SSD 硬體跑分數據，對於論文的內容會更加完善(也許成本是個問題)。